UN Comtrade: International Trade Data

NoSQL Lab Personal Project — Extraction Pilot

Table of contents

1 Summary and current scope **1**

2 Requirements and configuration **1**
2.1 API key 2
2.2 Pilot query settings 2

3 ELT process **3**
3.1 Extract: request a single year 3
3.2 Extract: collect the selected years 4
3.3 Inspect and validate the raw extraction 5
3.4 Pilot interpretation and next steps 6

4 Sources and method references **6**

5 PDF export and final submission checklist **7**

1 Summary and current scope

This notebook starts with an **extraction pilot** for the NoSQL personal project. It tests authenticated access and inspects the returned records before choosing the final research scope.

The pilot requests annual US exports of HS chapter **85** (electrical machinery and equipment) to the world for **2022 and 2023**. These are test settings, not the final research topic. Two aggregate observations are not enough for the final in-depth analysis.

Current workflow: configure → request JSON → collect raw records → inspect and validate.

Later: load into MongoDB → clean and model inside MongoDB → analyse with aggregation pipelines → add diagrams and export the report. No database writes occur in this version.

2 Requirements and configuration

Use a Python kernel with `requests` and `pandas` installed. Both are already listed in the project's requirements. The other imports belong to Python's standard library.

The course template uses `requests.get()`, `.json()`, a list of records, and a loop with a delay. We keep that approach. Authentication, query parameters, and response fields follow the

UN Comtrade documentation. Timeouts and bounded rate-limit retries are small reliability additions.

```
import os
import time
from datetime import datetime, timezone
from getpass import getpass
from pprint import pprint

import pandas as pd
import requests

# Keep notebook tables compact enough for reading and later PDF export.
pd.set_option("display.max_rows", 20)
pd.set_option("display.max_colwidth", 60)
```

2.1 API key

Use either your primary or secondary key, not both. The cell first checks the optional environment variable `UN_COMTRADE_API_KEY`. Otherwise, enter the key in the hidden prompt when running the notebook.

Never put the key in a code cell or display `api_key`, `headers`, or a request object's headers. The key is held in kernel memory; it is not intentionally saved in notebook text or outputs. On a kernel restart, enter it again unless the environment variable is configured.

```
# Read the secret without writing it into the notebook source.
api_key = os.environ.get("UN_COMTRADE_API_KEY", "").strip()
if not api_key:
    api_key = getpass("UN Comtrade API key: ").strip()

if not api_key:
    raise ValueError("An API key is required for this authenticated endpoint.")

# A header keeps the key out of the query URL.
headers = {"Ocp-Apim-Subscription-Key": api_key}
print("API key loaded. Its value is not displayed.")
```

2.2 Pilot query settings

Setting	Meaning
`C / A / HS` in the endpoint	Goods, annual frequency, original HS classification
`reporterCode = 842`	United States, the country reporting the trade
`partnerCode = 0`	World total, not a particular destination
`cmdCode = "85"`	One two-digit commodity chapter; keep codes as strings
`flowCode = "X"`	Exports

Setting	Meaning
`partner2Code = 0, customsCode = "C00", motCode = 0`	Aggregate selections for the additional dimensions
`includeDesc = "true"`	Request readable descriptions alongside codes
`maxRecords = 500`	Conservative request cap, not a guarantee of completeness

We request one year at a time. The original `HS` classification can resolve to different editions across countries or years; inspect the returned `classificationCode` before comparing them. The final project may instead need a fixed edition.

```
API_URL = "https://comtradeapi.un.org/data/v1/get/C/A/HS"
YEARS = ["2022", "2023"]
MAX_RECORDS = 500
REQUEST_DELAY = 5

# These filters describe one aggregate observation per year.
base_params = {
    "reporterCode": 842,
    "partnerCode": 0,
    "partner2Code": 0,
    "cmdCode": "85",
    "flowCode": "X",
    "customsCode": "C00",
    "motCode": 0,
    "maxRecords": MAX_RECORDS,
    "includeDesc": "true"
}
```

3 ELT process

3.1 Extract: request a single year

`requests.get()` downloads the response. `.raise_for_status()` stops on an HTTP error, and `.json()` converts JSON into Python dictionaries and lists. Comtrade's actual trade records are in `payload["data"]`, rather than the cocktail template's `data["drinks"]`.

The function retries HTTP **429** (rate limit) at most twice after the first attempt. Other HTTP failures stop immediately. For **401/403**, check the key and account access rather than repeatedly retrying. A response reaching the record cap is rejected until the query is narrowed or a documented larger extraction strategy is chosen.

```
def extract_year(year):
    params = base_params.copy()
    params["period"] = year

    for attempt in range(3):
        response = requests.get(
            API_URL, headers=headers, params=params, timeout=30
```

```
        )

        # Pause and retry only when the service reports a rate limit.
        if response.status_code == 429 and attempt < 2:
            wait_seconds = 15 * (attempt + 1)
            print(f"Rate limit for {year}; retrying in {wait_seconds} seconds.")
            time.sleep(wait_seconds)
            continue

        response.raise_for_status()
        payload = response.json()

        if not isinstance(payload, dict):
            raise ValueError("Expected a JSON object from Comtrade.")
        if payload.get("error"):
            raise ValueError("Comtrade reported an API error; review the query and documenta
        if not isinstance(payload.get("data"), list):
            raise ValueError("Expected a list of records in the API's data field.")

        records = payload["data"]
        if len(records) >= MAX_RECORDS:
            raise ValueError("The record cap was reached; completeness is not established.")

        return records
```

3.2 Extract: collect the selected years

Like the course template, we loop over query values, add the returned documents to a list, and pause between requests. Here the query values are years, not the first letters of drink names.

`list_trade` preserves the returned trade fields. `extraction_log` separately records when each query was made and how many records it returned. Re-running this cell starts a fresh in-memory list; it does not append to a database or file.

```
list_trade = []
extraction_log = []

for i, year in enumerate(YEARS):
    print(f"Downloading {year} ({i + 1}/{len(YEARS)})")
    extracted_at = datetime.now(timezone.utc).isoformat()
    records = extract_year(year)

    # This exact pilot query should return one aggregate record per year.
    if len(records) != 1:
        raise ValueError(
            f"Expected one record for {year}, received {len(records)}. "
            "Check availability and query scope before continuing."
        )

    list_trade.extend(records)
    extraction_log.append({
```

```
        "year_requested": year,
        "records_received": len(records),
        "extracted_at_utc": extracted_at,
        "endpoint": API_URL,
        "query": {**base_params, "period": year}
    })

    if i < len(YEARS) - 1:
        time.sleep(REQUEST_DELAY)

print(f"Extraction finished: {len(list_trade)} raw records.")
```

3.3 Inspect and validate the raw extraction

First inspect one full record, as in the course template. Then check that the returned dimensions match the requested pilot. These are extraction checks, not the final MongoDB cleaning step.

We intentionally keep missing measurements and quality flags unchanged. A zero quantity with an unavailable unit is not automatically a meaningful measured zero. World and commodity totals must not later be summed together with their component records.

```
# Inspect a raw trade document, not the secret-bearing request headers.
pprint(list_trade[0])
```

```
expected_fields = {
    "reporterCode": base_params["reporterCode"],
    "partnerCode": base_params["partnerCode"],
    "partner2Code": base_params["partner2Code"],
    "cmdCode": base_params["cmdCode"],
    "flowCode": base_params["flowCode"],
    "customsCode": base_params["customsCode"],
    "motCode": base_params["motCode"]
}

for record, year in zip(list_trade, YEARS):
    if str(record.get("period")) != year:
        raise ValueError("A returned period does not match the request.")
    for field, expected in expected_fields.items():
        if record.get(field) != expected:
            raise ValueError(f"Unexpected value for {field}; review the response.")
    if record.get("primaryValue") is None:
        raise ValueError("A requested record has no primary trade value.")
    if not record.get("classificationCode"):
        raise ValueError("A returned record has no classification edition.")

editions = {record["classificationCode"] for record in list_trade}
if len(editions) != 1:
    raise ValueError("Multiple HS editions returned; resolve comparability before analysis.'

print("Pilot dimensions and trade-value presence checked.")
print("Returned HS edition:", sorted(editions))
```

```
# Pandas is only for inspection here; list_trade remains the raw input for MongoDB.
preview_columns = [
    "period", "reporterDesc", "partnerDesc", "cmdCode",
    "classificationCode", "primaryValue", "netWgt", "isNetWgtEstimated"
]
df_preview = pd.DataFrame(list_trade)
df_preview.reindex(columns=preview_columns)
```

```
# The log contains query settings and timestamps, but no API key.
pd.DataFrame(extraction_log)[
    ["year_requested", "records_received", "extracted_at_utc"]
]
```

3.4 Pilot interpretation and next steps

A successful run should produce two raw observations, one per selected year, with matching query dimensions and one returned HS edition. This validates a narrow access path; it does not establish completeness for a larger query or provide the final project's conclusions.

Before expanding, choose the reporting country, research questions, commodity level, time range, and partner scope. Revisit the one-record-per-year check if those settings change. Keep a single commodity level and separate World totals from partner breakdowns to avoid double-counting.

Next, load an agreed raw dataset into MongoDB with a repeatable import strategy. Perform the main cleaning, embedded-document modeling, and analytical aggregations inside MongoDB. The final submission still needs the class diagram, a second image, conclusions, learnings, and a checked PDF.

4 Sources and method references

- UN Comtrade API documentation: endpoints, query parameters, and authentication.
- UN Comtrade API guide: public preview versus authenticated access.
- Subscription plans: check current free-tier limits before a larger extraction.
- Commodity classification and conversion issues: HS revisions and comparability.
- Trade valuation: interpretation of primary trade values and import/export valuation differences.
- Course project template: the `requests` → JSON → list → load workflow.
- Local course notes: PyMongo insertion, document modeling, and MongoDB transformations for subsequent stages.

Trade values are in current US dollars. They are not inflation-adjusted. Download timestamps are recorded when the extraction cell runs; source data may be revised.

5 PDF export and final submission checklist

The first Markdown cell contains Quarto YAML settings. Quarto supplies the title, table of contents, and section numbering, so headings do not contain manually typed numbers. Code remains visible to document the method. Rendering is configured **not to execute Python cells**: the PDF uses the notebook's saved outputs.

From a terminal opened in this notebook's folder, render with:

```
quarto render "UN_commodity.ipynb" --to pdf --no-execute
```

Before the final export:

- Replace the pilot subtitle with the final project subtitle and add your author details to the YAML header.
- Complete the analysis, conclusions, learnings, and source references; remove or rewrite this working checklist for the submitted report.
- Include at least two images, one being the class diagram. Use relative paths, readable labels, and captions.
- Keep result tables narrow, with clear units and sensible precision. A Pandas table that looks good in Jupyter may need adjustment for LaTeX PDF output.
- Restart the kernel, run all cells successfully in Jupyter, and save. Running the notebook performs the configured requests and, once implemented, database operations; inspect the rerun policy first.
- Check code and saved outputs for credentials, tracebacks, excessive logs, and unnecessary raw dumps. Hide verbose cells selectively for the PDF without removing reproducible code from the notebook.
- Render and inspect every page for clipped code or tables, broken images, duplicated numbering, and awkward page breaks. This pilot PDF has no data results until the extraction cells have actually run and their outputs have been saved.
- Package the final PDF, executed notebook, and image files in a ZIP. Exclude secret files and temporary render artifacts. Check that the notebook's image references work after extracting the ZIP.

Quarto and TinyTeX were available during the initial setup check. If rendering later becomes impractical, browser Print → Save as PDF is an assignment-approved fallback.